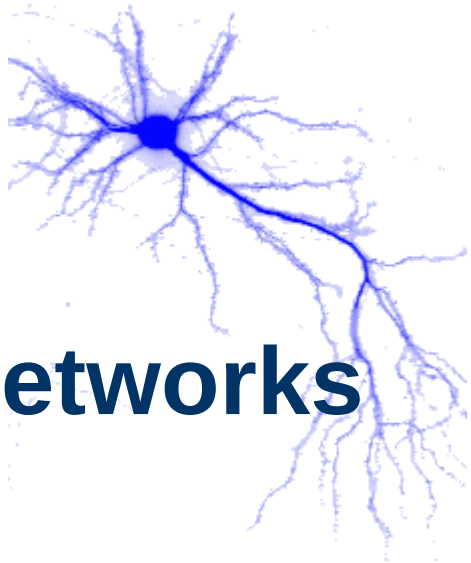
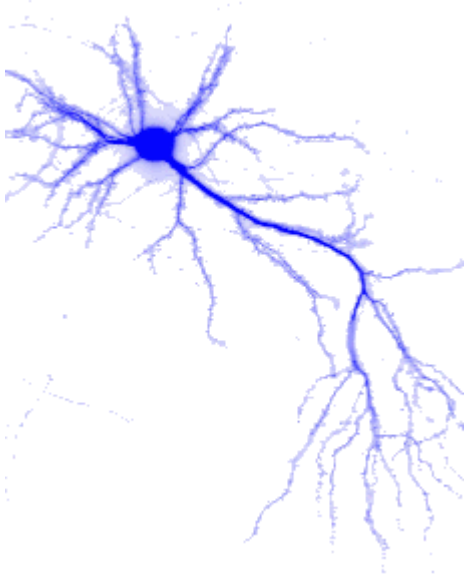
Introduction to Radial Basis Function Networks



Content

- Overview
- The Models of Function Approximator
- The Radial Basis Function Networks
- RBFN’s for Function Approximation
- Learning the Kernels



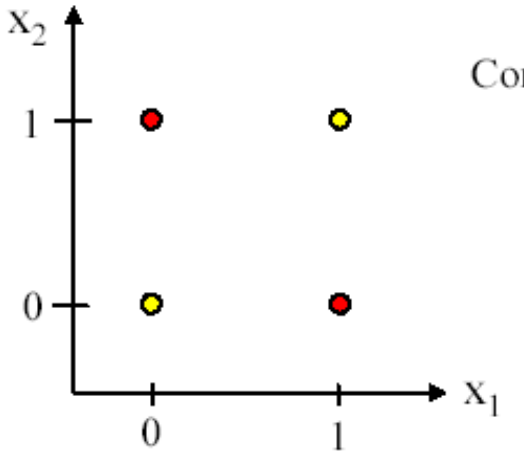
Overview

Cover's Theorem (1965)

(Based on the separability of patterns)

A complex pattern classification problem that is nonlinearly separable in a low dimensional space, is more likely to be linearly separable in a high dimensional space.

Example - XOR Problem

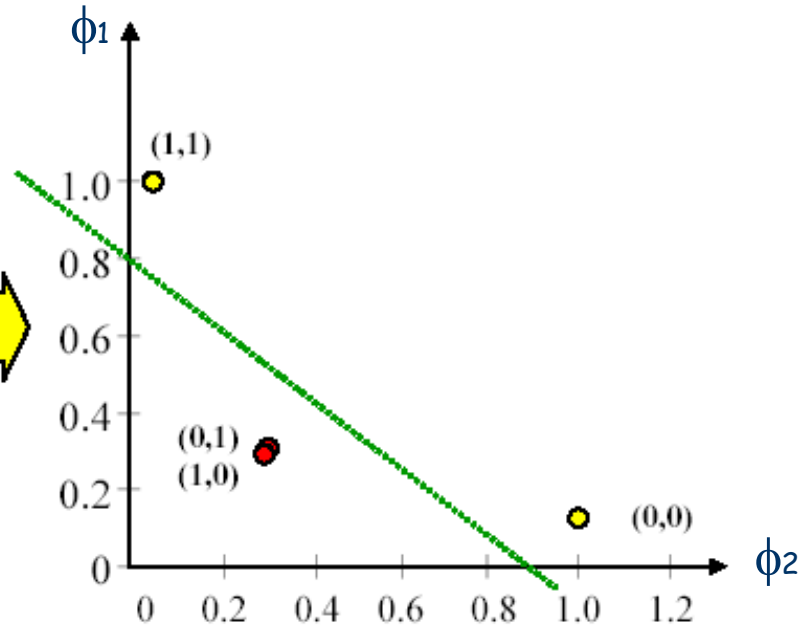


Consider the nonlinear functions to map the input vector **x** to the φ_1- φ_2 space

$$\mathbf{x}=[x_1\ x_2] \qquad \varphi_1(\mathbf{x}) = e^{-\|\mathbf{x}-\mathbf{t}_1\|^2} \qquad \mathbf{t}_1 = [1\ \ 1]^T$$

$$\varphi_2(\mathbf{x}) = e^{-\|\mathbf{x}-\mathbf{t}_2\|^2} \qquad \mathbf{t}_2 = [0\ \ 0]^T$$

Input x	$\varphi_1(x)$	$\varphi_2(x)$
(1,1)	1	0.1353
(0,1)	0.3678	0.3678
(1,0)	0.3678	0.3678
(0,0)	0.1353	1



The nonlinear φ function transformed a nonlinearly separable problem into a linearly separable one !!!

Typical Applications of NN

- Pattern Classification

$$l = f(\mathbf{x}) \qquad \mathbf{x} \in X \subset R^m \qquad l \in C \subset N$$

- Function Approximation

$$\mathbf{y} = f(\mathbf{x}) \qquad \mathbf{x} \in X \subset R^n \qquad \mathbf{y} \in Y \subset R^m$$

- Time-Series Forecasting

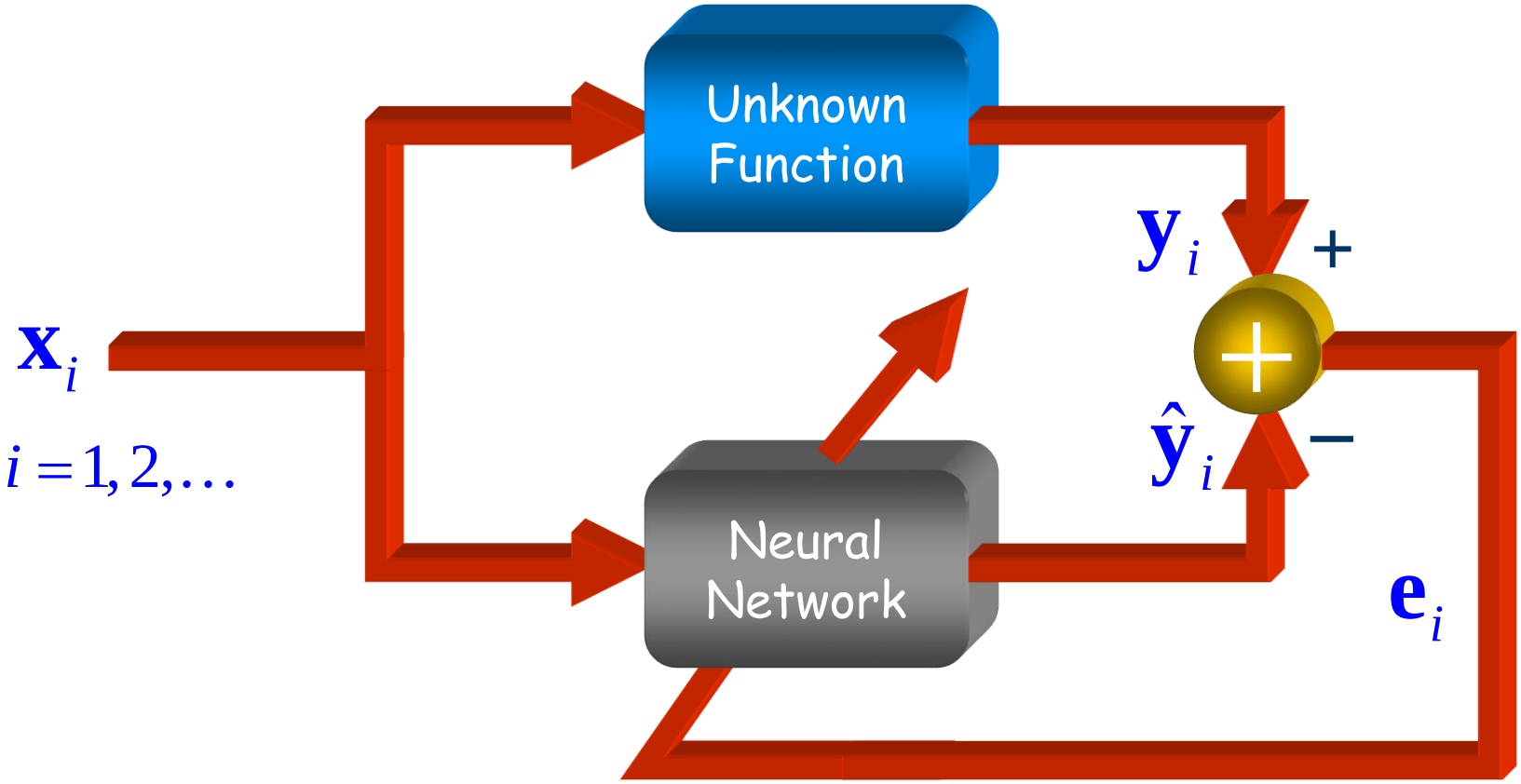
$$\mathbf{x}(t) = f(\mathbf{x}_{t-1}, \mathbf{x}_{t-2}, \mathbf{x}_{t-3}, \ldots)$$

Function Approximation

Unknown $f : X \to Y$

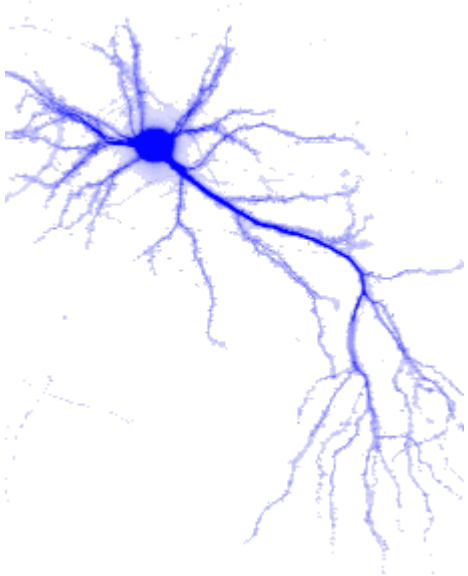
Approximator $\hat{f} : X \to Y$

Supervised Learning



Neural Networks as Universal Approximators

- Feedforward neural networks with a single hidden layer of sigmoidal units are capable of approximating uniformly any continuous multivariate function, to any desired degree of accuracy.
  - Hornik, K., Stinchcombe, M., and White, H. (1989). "Multilayer Feedforward Networks are Universal Approximators," Neural Networks, 2(5), 359-366.
- Like feedforward neural networks with a single hidden layer of sigmoidal units, it can be shown that RBF networks are universal approximators.
  - Park, J. and Sandberg, I. W. (1991). "Universal Approximation Using Radial-Basis-Function Networks," Neural Computation, 3(2), 246-257.
  - Park, J. and Sandberg, I. W. (1993). "Approximation and Radial-Basis-Function Networks," Neural Computation, 5(2), 305-316.



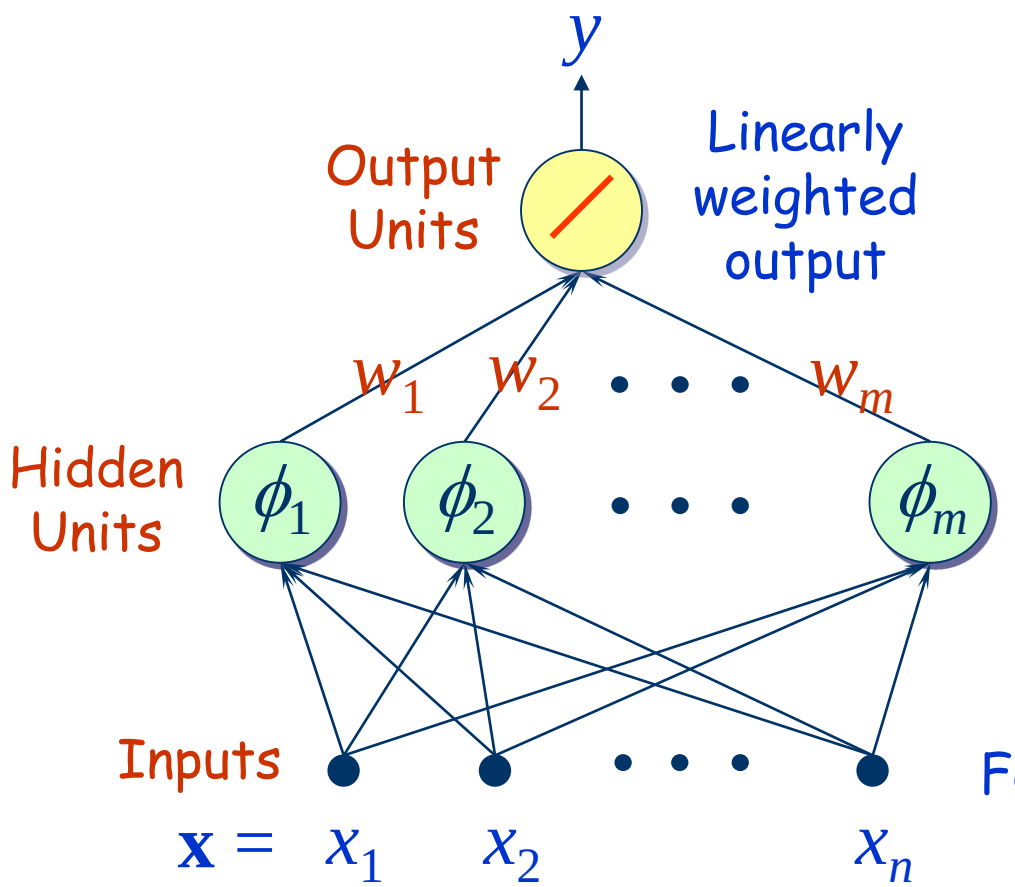
The Model of Function Approximator

Linear Models

$$f(\mathbf{x}) = \sum_{i=1}^{m} \overbrace{w_i}^{\text{Weights}} \underbrace{\phi_i(\mathbf{x})}_{\text{Fixed Basis Functions}}$$

$$f(\mathbf{x}) = \sum_{i=1}^{m} w_i \phi_i(\mathbf{x})$$

Linear Models



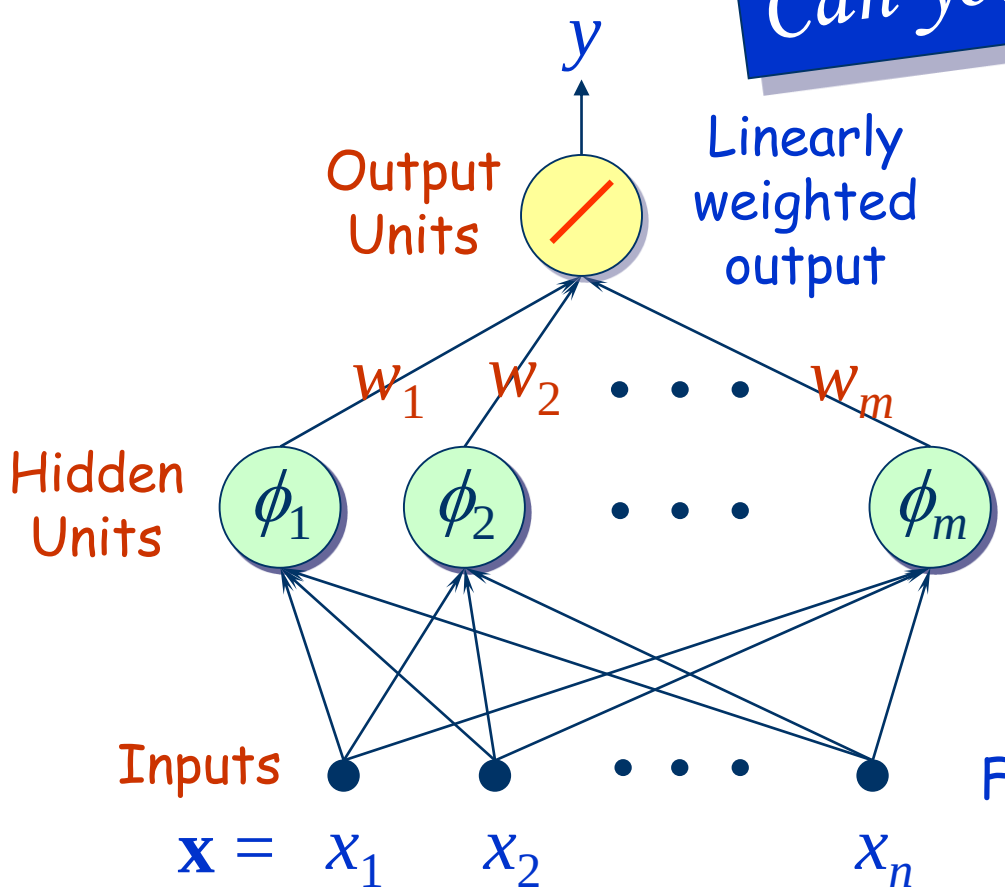
- Decomposition
- Feature Extraction
- Transformation

Feature Vectors

$$f(\mathbf{x}) = \sum_{i=1}^{m} w_i \phi_i(\mathbf{x})$$

Linear Models

Can you say some bases?



- Decomposition
- Feature Extraction
- Transformation

Feature Vectors

$$f(\mathbf{x}) = \sum_{i=1}^{m} w_i \phi_i(\mathbf{x})$$

Example Linear Models

Are they orthogonal bases?

- Polynomial

$$f(x) = \sum_i w_i x^i \qquad \phi_i(x) = x^i, \quad i = 0,1,2,\ldots$$

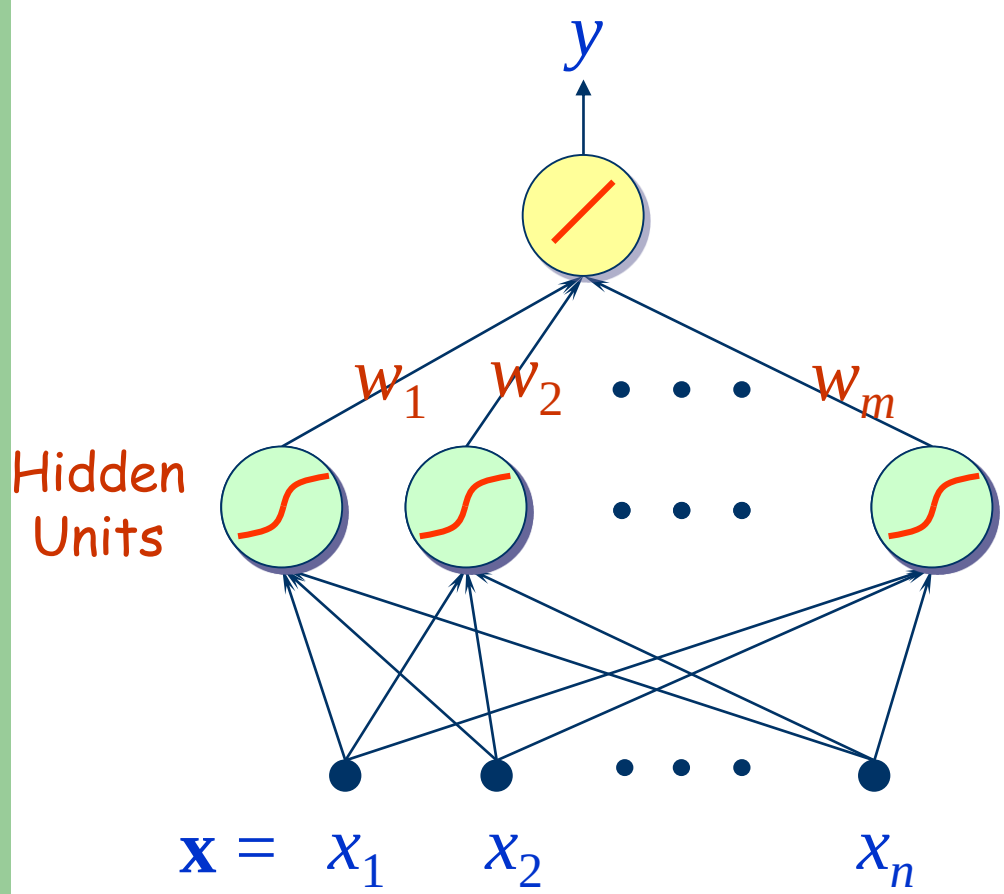
- Fourier Series

$$f(x) = \sum_k w_k \exp\left(j2k\omega_0 x\right)$$

$$\phi_k(x) = \exp\left(j2k\omega_0 x\right), \quad k = 0,1,2,\ldots$$

$$f(\mathbf{x}) = \sum_{i=1}^{m} w_i \phi_i(\mathbf{x})$$

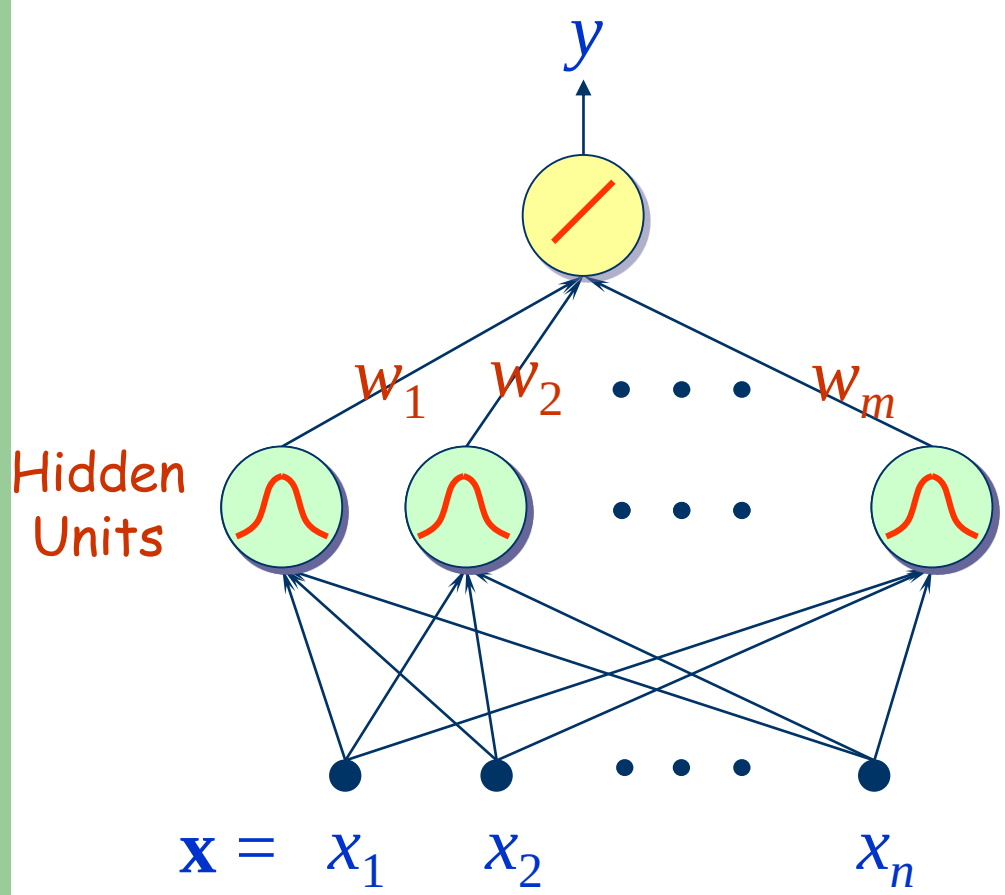
Single-Layer Perceptrons as Universal Aproximators



With sufficient number of sigmoidal units, it can be a universal approximator.

$$f(\mathbf{x}) = \sum_{i=1}^{m} w_i \phi_i(\mathbf{x})$$

Radial Basis Function Networks as Universal Aproximators

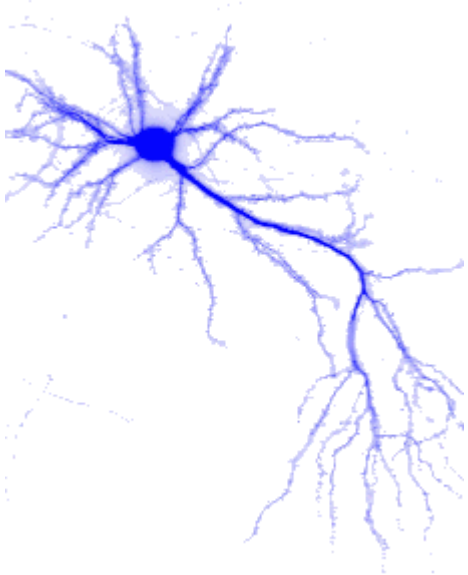


With sufficient number of radial-basis-function units, it can also be a universal approximator.

$$f(\mathbf{x}) = \sum_{i=1}^{m} w_i \phi_i(\mathbf{x})$$

Non-Linear Models

$$f(\mathbf{x}) = \sum_{i=1}^{m} \overbrace{w_i}^{\text{Weights}} \underbrace{\phi_i(\mathbf{x})}_{\text{Adjusted by the Learning process}}$$



The Radial Basis Function Networks

Radial Basis Function Networks

Radial Basis Functions are feed-forward networks consisting of

- ❖A hidden layer of radial kernels and
- ❖An output layer of linear neurons

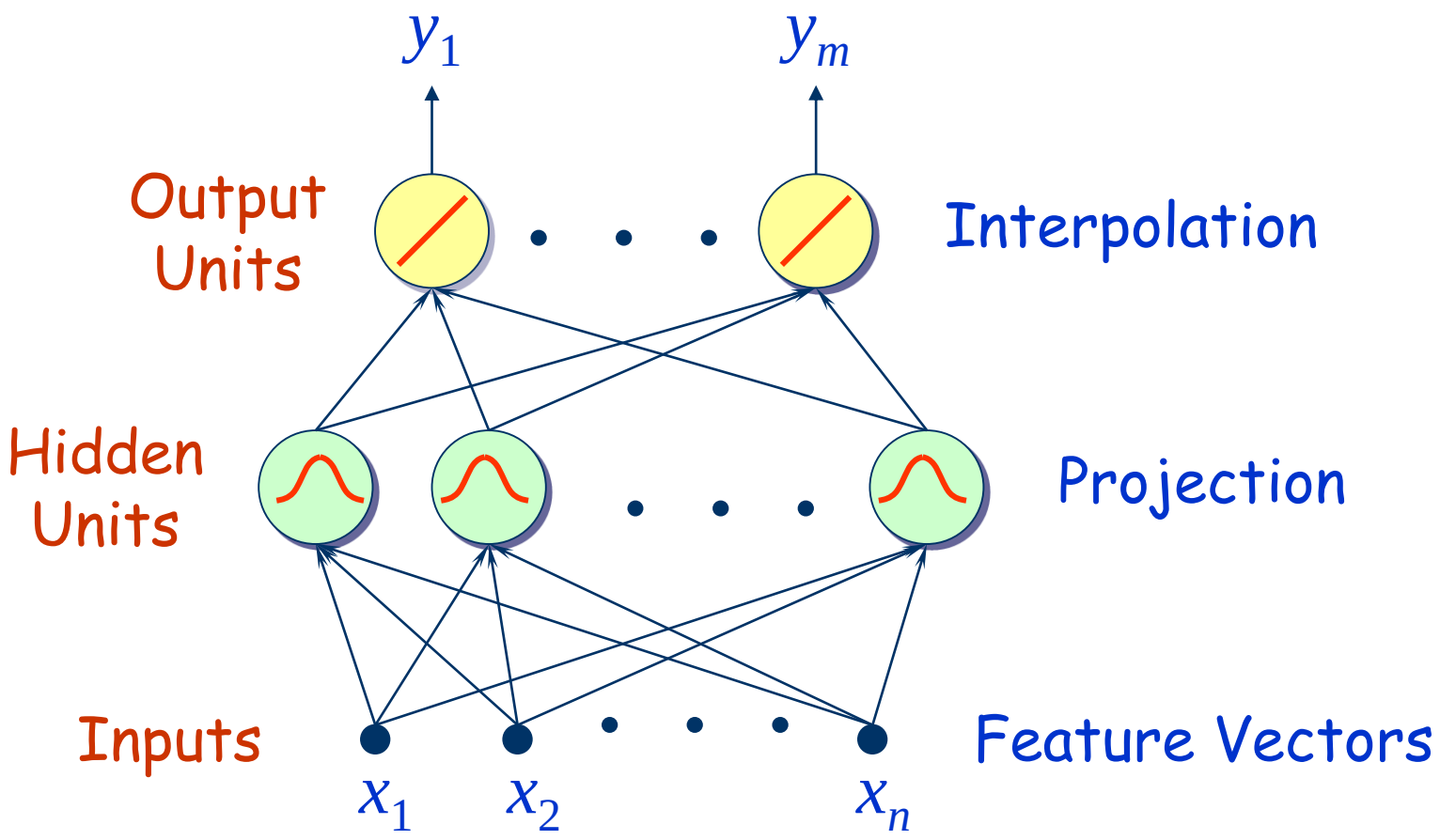
The two layers in an RBF carry entirely different roles [Haykin, 1999]

- ❖The hidden layer performs a **non-linear transformation** of input space
- ❖The resulting hidden space is typically of **higher dimensionality** than the input space
- ❖The output layer performs **linear regression** to predict the desired targets

As a function approximator

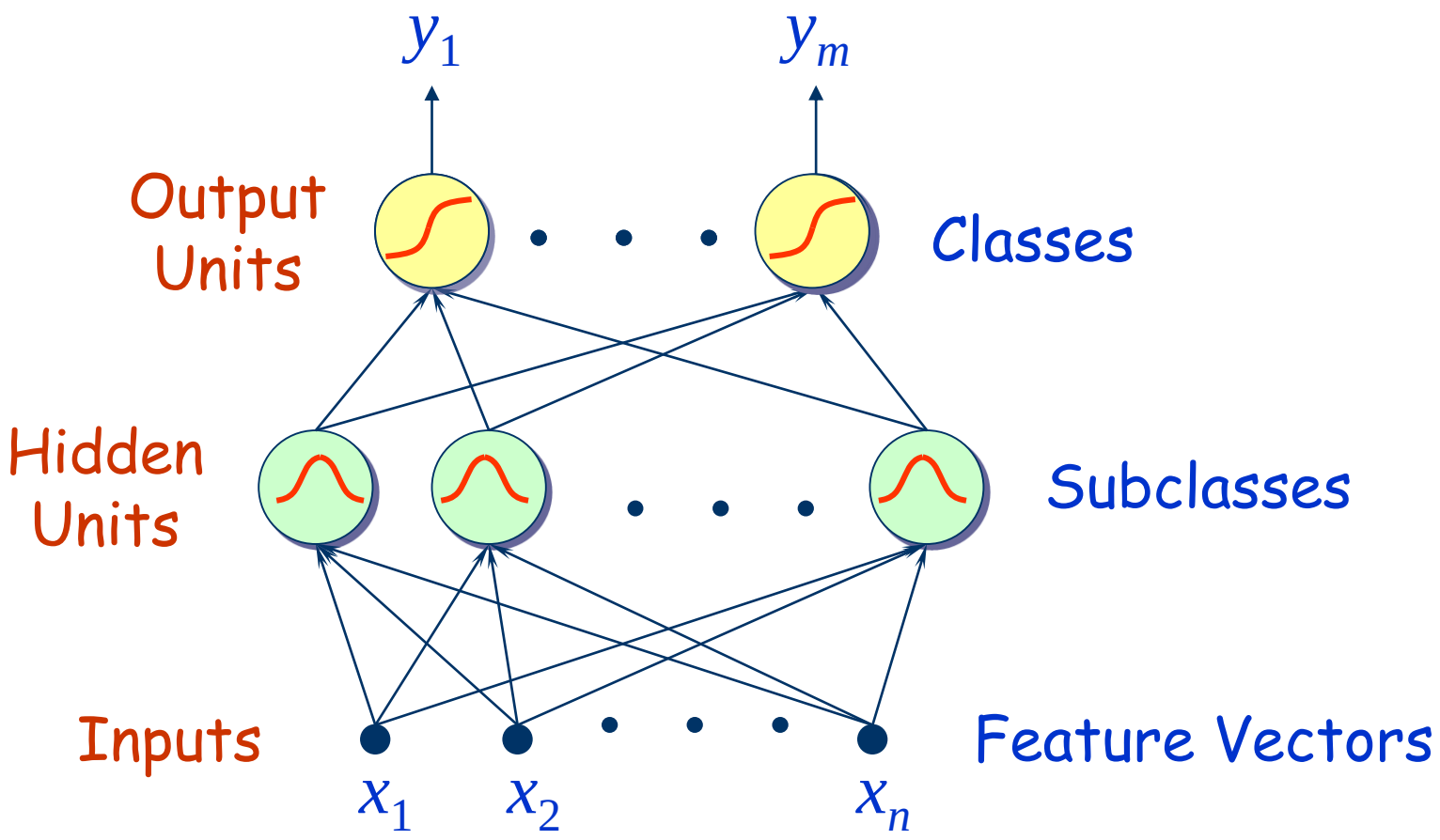
$$\mathbf{y} = \mathbf{f}(\mathbf{x})$$

The Topology of RBF



As a pattern classifier.

The Topology of RBF



Network Parameters

- φ **: The radial basis function for the hidden layer.**

  This is a simple nonlinear mapping function (typically Gaussian) that transforms the ***d***- dimensional input patterns to a (typically higher) ***H***-dimensional space. The complex decision boundary will be constructed from linear combinations (weighted sums) of these simple building blocks.

- u_{ji} **: The weights joining the first to hidden layer.** These weights constitute the **center points** of the radial basis functions. Also called **prototypes** of data.

Network Parameters

- σ **: The spread constant(s).** These values determine the spread (extend) of each radial basis function.

- w_{jk}**: The weights joining hidden and output layers**. These are the weights which are used in obtaining the linear combination of the radial basis functions. They determine the relative amplitudes of the RBFs when they are combined to form the complex function.

- $\|x - u_j\|$ **:** the Euclidean distance between the input x and the prototype vector u_{ji}. Activation of the hidden unit is determined according to this distance through φ.

Typical Radial Functions

- Gaussian

$$\phi(r) = e^{-\frac{r^2}{2\sigma^2}} \quad \sigma > 0 \quad \text{and} \quad r \in \Re$$

- Hardy Multiquadratic

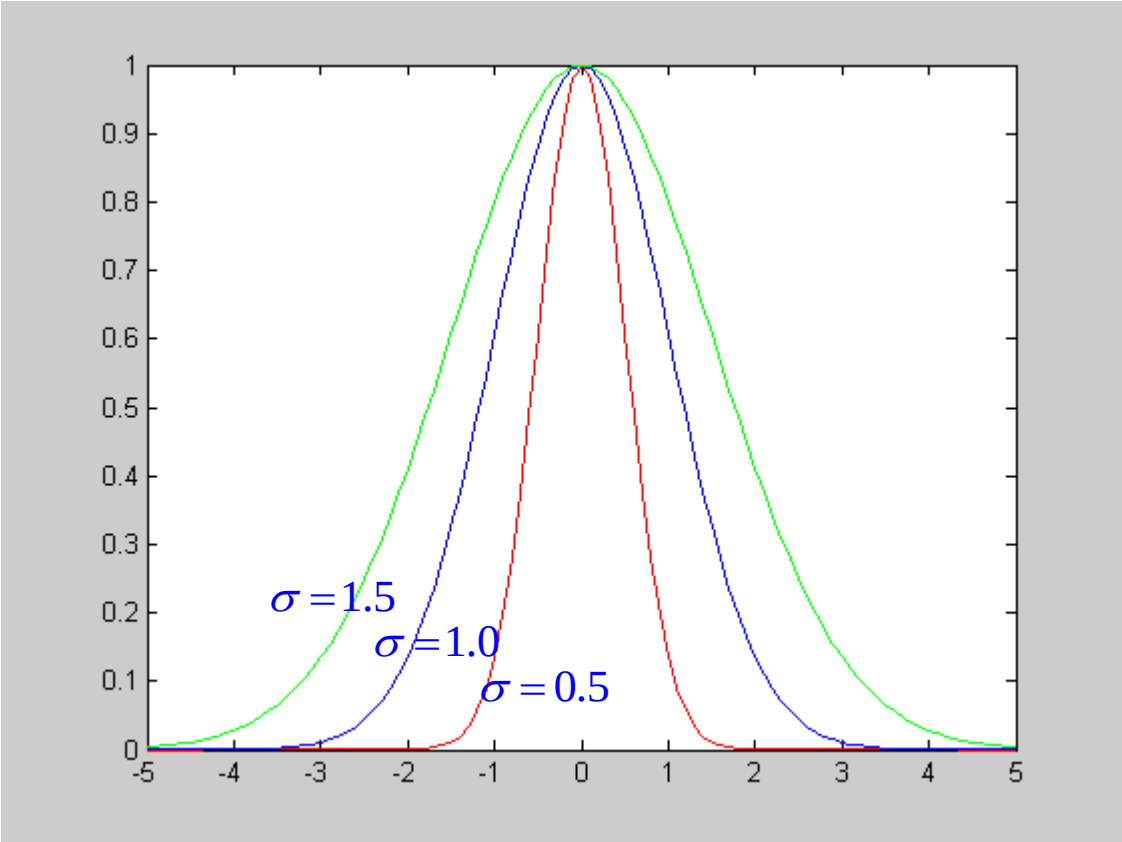
$$\phi(r) = \sqrt{r^2 + c^2} \Big/ c \quad c > 0 \quad \text{and} \quad r \in \Re$$

- Inverse Multiquadratic

$$\phi(r) = c \Big/ \sqrt{r^2 + c^2} \quad c > 0 \quad \text{and} \quad r \in \Re$$

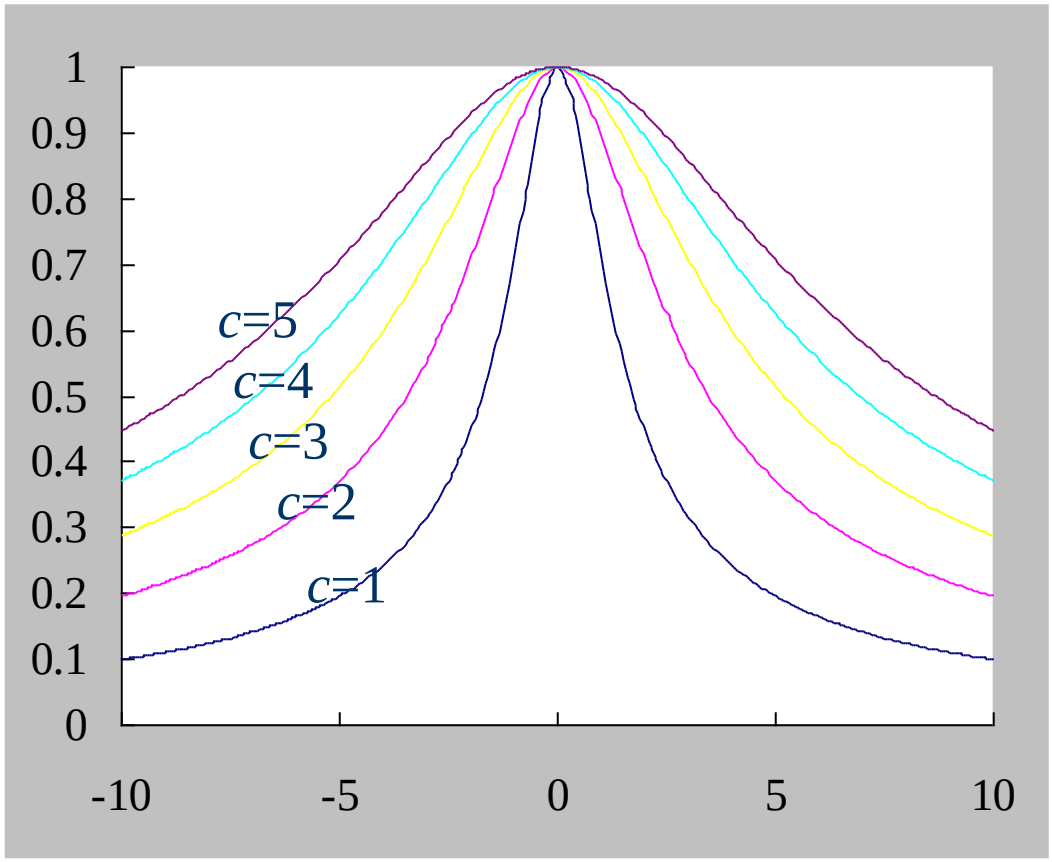
$$\phi(r) = e^{-\frac{r^2}{2\sigma^2}} \quad \sigma > 0 \quad \text{and} \quad r \in \Re$$

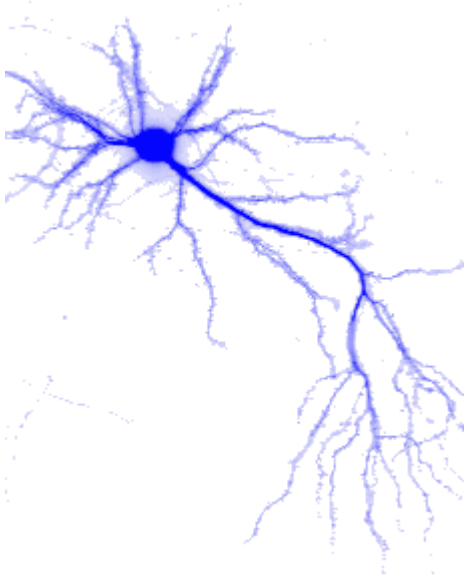
Gaussian Basis Function (σ=0.5,1.0,1.5)



$$\phi(r) = c\Big/\sqrt{r^2 + c^2} \qquad c > 0 \quad \text{and} \quad r \in \Re$$

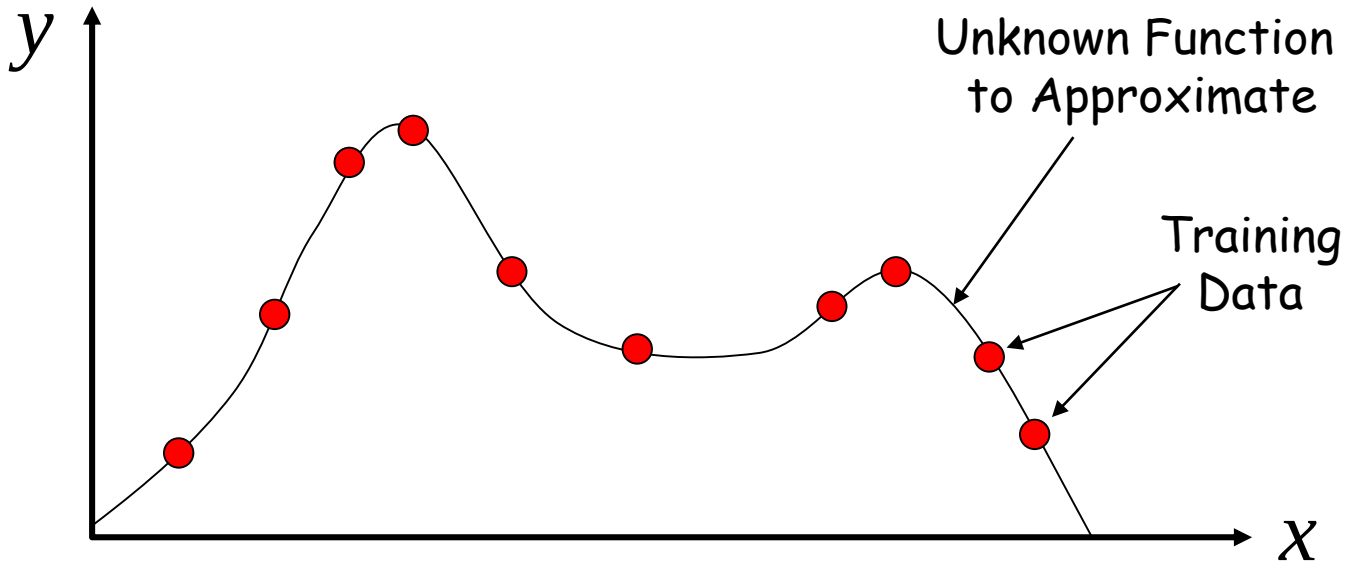
Inverse Multiquadratic



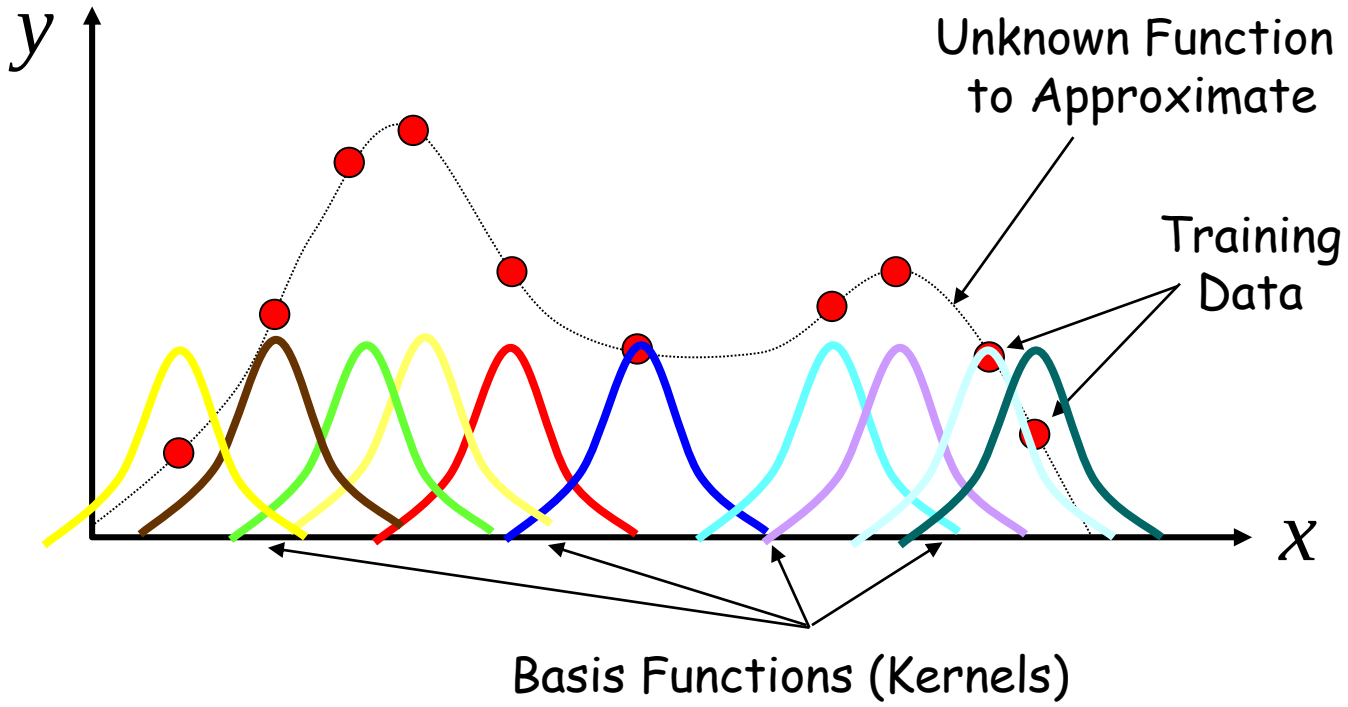


RBFN’s for Function Approximation

The idea

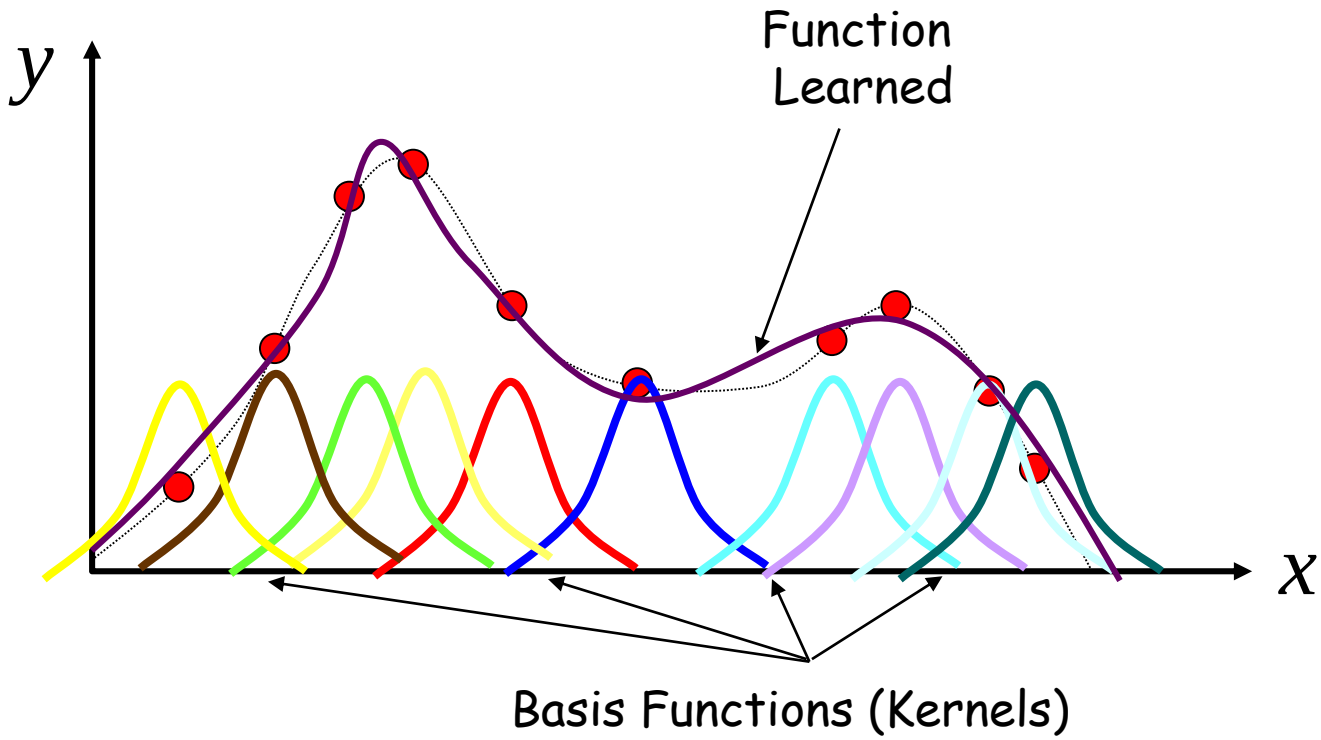


The idea


y
x
Unknown Function
to Approximate
Training
Data
Basis Functions (Kernels)

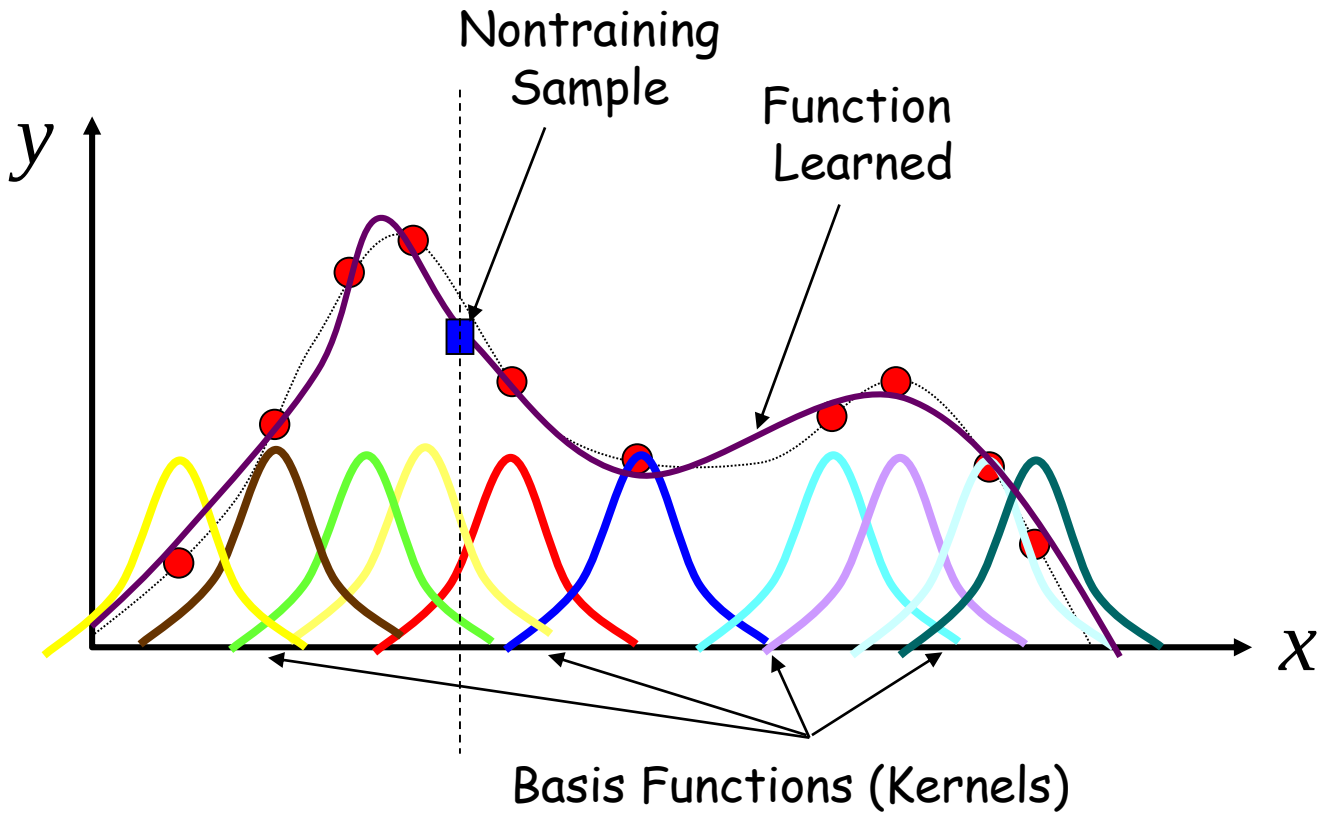
The idea

$$y = f(\mathbf{x}) = \sum_{i=1}^{m} w_i \phi_i(\mathbf{x})$$



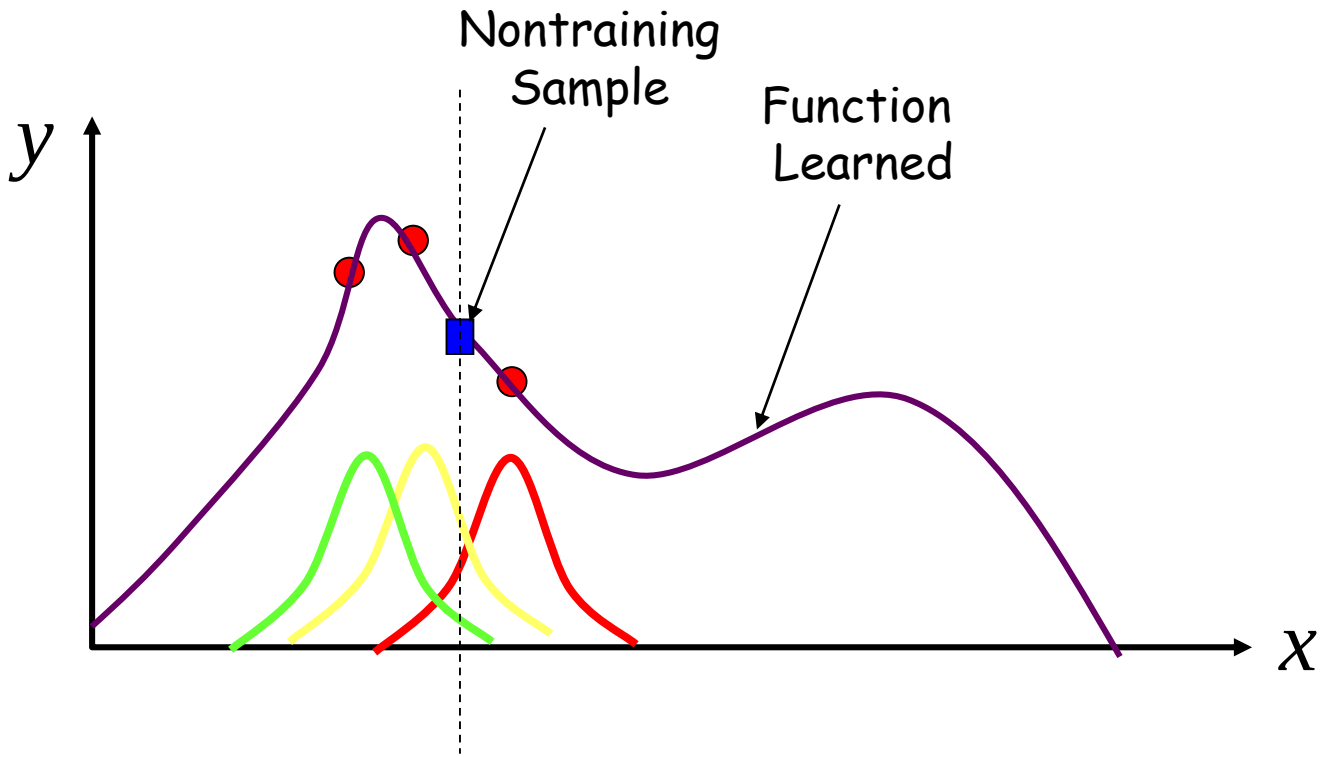
The idea

$$y = f(\mathbf{x}) = \sum_{i=1}^{m} w_i \phi_i(\mathbf{x})$$



The idea

$$y = f(\mathbf{x}) = \sum_{i=1}^{m} w_i \phi_i(\mathbf{x})$$



Radial Basis Function Networks as Universal Aproximators

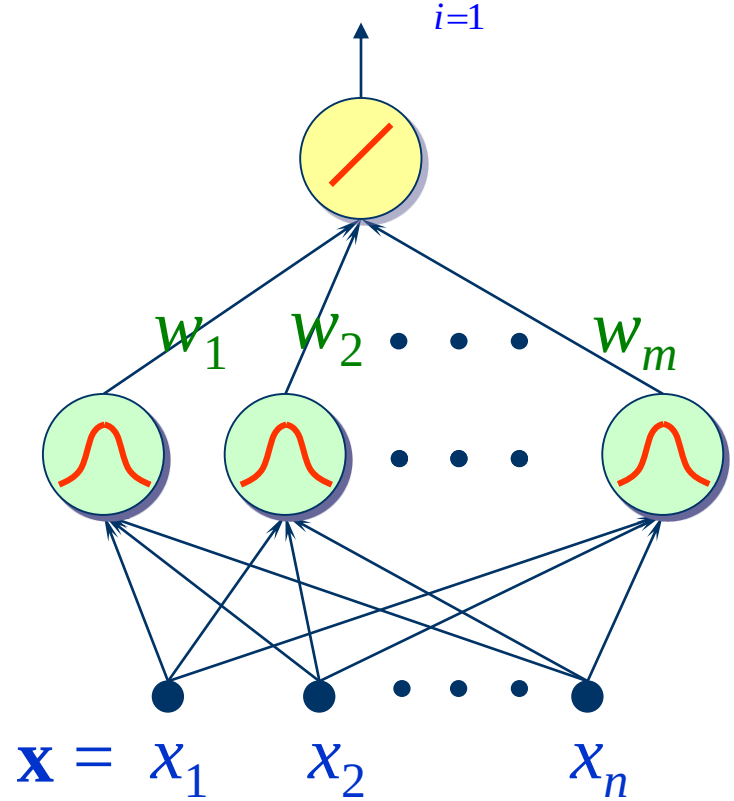
Training set $\mathcal{T} = \left\{ \left(\mathbf{x}^{(k)}, y^{(k)} \right) \right\}_{k=1}^{p}$

Goal $y^{(k)} \approx f\left(\mathbf{x}^{(k)}\right)$ for all k

$$\min E = \frac{1}{2} \sum_{k=1}^{p} \left[y^{(k)} - f\left(\mathbf{x}^{(k)}\right) \right]^2$$

$$= \frac{1}{2} \sum_{k=1}^{p} \left[y^{(k)} - \sum_{i=1}^{m} w_i \phi_i \left(\mathbf{x}^{(k)}\right) \right]^2$$

$$y = f(\mathbf{x}) = \sum_{i=1}^{m} w_i \phi_i(\mathbf{x})$$



Learn the Optimal Weight Vector

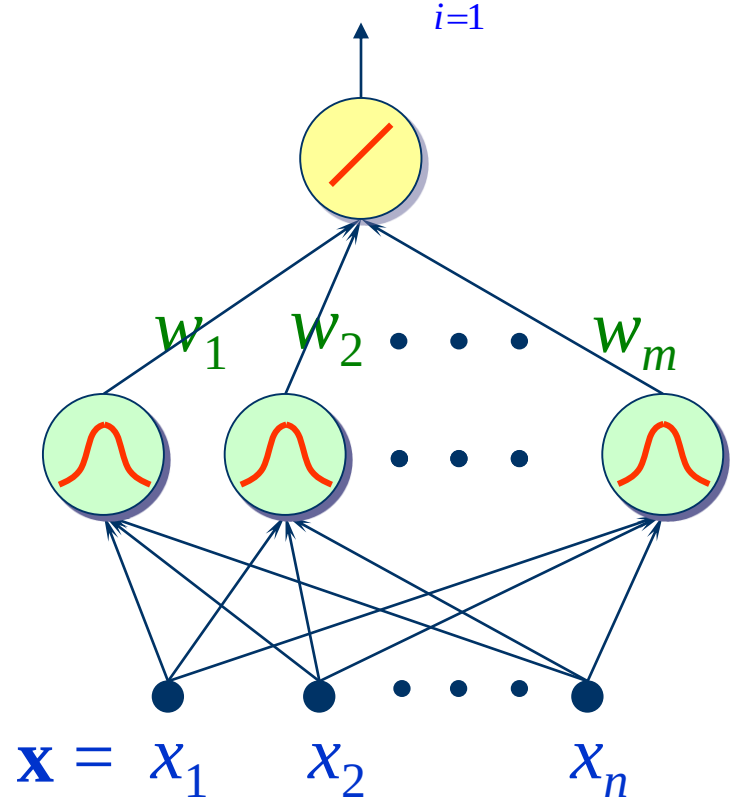
Training set $\mathcal{T} = \left\{ \left(\mathbf{x}^{(k)}, y^{(k)} \right) \right\}_{k=1}^{p}$

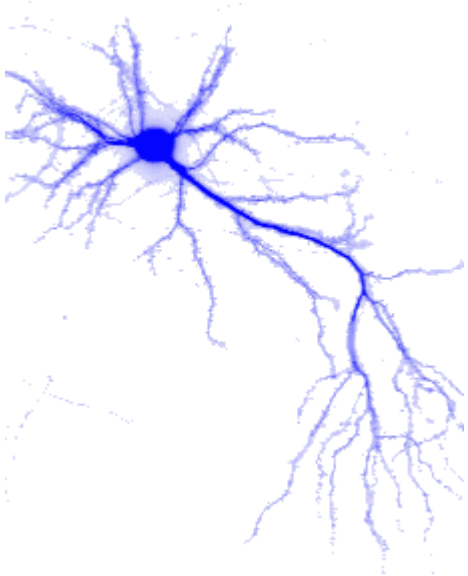
Goal $y^{(k)} \approx f\left(\mathbf{x}^{(k)}\right)$ for all k

$$\min E = \frac{1}{2} \sum_{k=1}^{p} \left[y^{(k)} - f\left(\mathbf{x}^{(k)}\right) \right]^2$$

$$= \frac{1}{2} \sum_{k=1}^{p} \left[y^{(k)} - \sum_{i=1}^{m} w_i \phi_i \left(\mathbf{x}^{(k)}\right) \right]^2$$

$$y = f(\mathbf{x}) = \sum_{i=1}^{m} w_i \phi_i(\mathbf{x})$$





Learning the Kernels

How to Train?

- There are various approaches for training RBF networks.
  - **Approach 1: Exact RBF –** Guarantees correct classification of all training data instances. Requires N hidden layer nodes, one for each training instance. No iterative training is involved. RBF centers (**u**) are fixed as training data points, spread as variance of the data, and **w** are obtained by solving a set of linear equations
  - **Approach 2: Fixed centers selected at random**. Uses $H<N$ hidden layer nodes. No iterative training is involved. Spread is based on Euclidean metrics, **w** are obtained by solving a set of linear equations.
  - **Approach 3: Centers are obtained from unsupervised learning (clustering).** Spreads are obtained as variances of clusters, **w** are obtained through LMS algorithm. Clustering (k-means) and LMS are iterative. This is the most commonly used procedure. Typically provides good results.
  - **Approach 4: All unknowns are obtained from supervised learning**.

Approach 1

Exact RBF

- The first layer weights **u** are set to the training data; $\mathbf{U}=\mathbf{X}^T$. That is the gaussians are centered at the training data instances.
- The spread is chosen as $\sigma = \frac{d_{\max}}{\sqrt{2N}}$, where d_{max} is the maximum Euclidean distance between any two centers, and N is the number of training data points. Note that $H=N$, for this case.
- The output of the kth RBF output neuron is then

$$y_k = \sum_{j=1}^{N} w_{kj} \cdot \varphi\left\|\mathbf{x}-\mathbf{u}_j\right\|$$

Multiple outputs

$$y = \sum_{j=1}^{N} w_j \cdot \varphi\left\|\mathbf{x}-\mathbf{u}_j\right\|$$

Single output

- During training, we want the outputs to be equal to our desired targets. Without loss of any generality, assume that we are approximating a single dimensional function, and let the unknown true function be $f(\mathbf{x})$. The desired output for each input is then $d_i=f(\mathbf{x}_i)$, $i=1, 2, \ldots, N$.

Approach 1 (Cont.)

- We then have a set of linear equations, which can be represented in the matrix form:

$$y = \sum_{j=1}^{N} w_j \cdot \varphi\left\|\mathbf{x} - \mathbf{u}_j\right\| \Rightarrow \begin{bmatrix} \varphi_{11} & \varphi_{12} & \cdots & \varphi_{1N} \\ \varphi_{21} & \varphi_{22} & \cdots & \varphi_{2N} \\ \vdots & \vdots & \vdots & \vdots \\ \varphi_{N1} & \varphi_{N2} & \cdots & \varphi_{NN} \end{bmatrix} \cdot \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_N \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ \vdots \\ d_N \end{bmatrix}$$

$$\varphi_{ij} = \varphi\left\|\mathbf{x}_i - \mathbf{x}_j\right\|, \quad (i, j) = 1,2,\ldots, N$$

Define:

$$\mathbf{d} = [d_1, d_2, \cdots d_N]^T$$

$$\mathbf{w} = [w_1, w_2, \cdots w_N]^T$$

$$\Phi = \{\varphi_{ij} \mid (i, j) = 1,2,\ldots, N\}$$

$$\Rightarrow \quad \Phi \cdot \mathbf{w} = \mathbf{d}$$

$$\boxed{\mathbf{w} = \Phi^{-1}\mathbf{d}}$$

Is this matrix always invertible?

Approach 1 (Cont.)

- Michelli's Theorem (1986)

  - If $\{\mathbf{x}_i\}_{i=1}^{N}$ are a distinct set of points in the d-dimensional space, then the N by N ***interpolation matrix*** Φ with elements obtained from radial basis functions $\varphi_{ij} = \varphi\|\mathbf{x}_i - \mathbf{x}_j\|$ is nonsingular, and hence can be inverted!

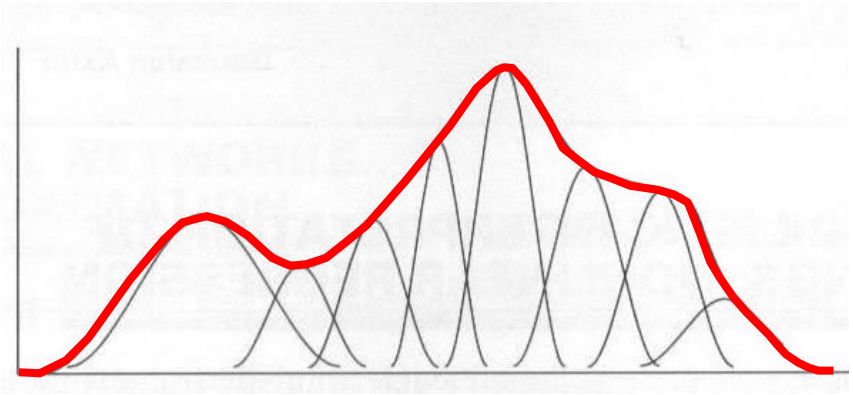
  - Note that the theorem is valid regardless the value of N, the choice of the RBF (as long as it is an RBF), or what the data points may be, as long as they are distinct!

Approach 1 (Cont.)

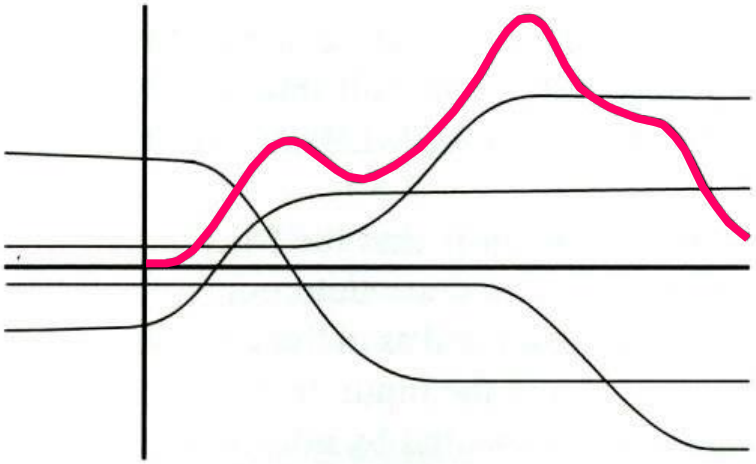
- The Gaussian is the most commonly used RBF (why…?).
- Note that

$$\text{as } r \to \infty, \quad \varphi(r) \to 0$$

  - Gaussian RBFs are ***localized functions !*** unlike the sigmoids used by MLPs



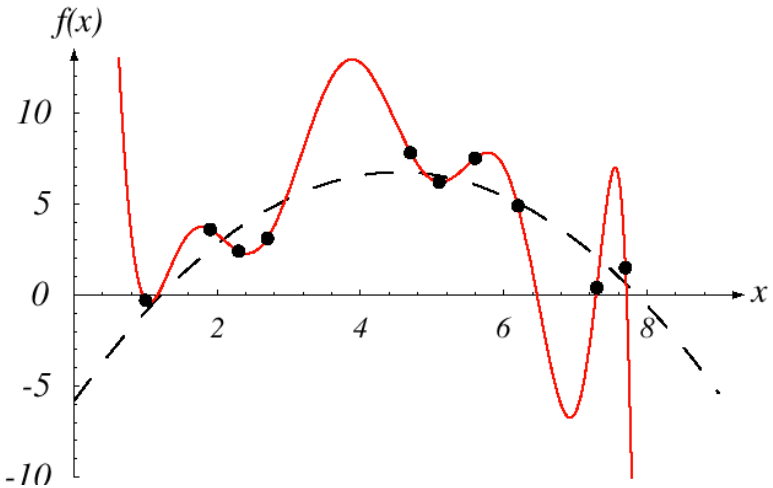
Using Gaussian radial basis functions



Using sigmoidal radial basis functio

Exact RBF Properties

- Using localized functions typically makes RBF networks more suitable for function approximation problems.
- Since first layer weights are set to input patterns, second layer weights are obtained from solving linear equations, and spread is computed from the data, no iterative training is involved !!!
- Guaranteed to correctly classify all training data points!
- However, since we are using as many receptive fields as the number of data, the solution is over determined, if the underlying physical process does not have as many degrees of freedom ➔ Overfitting!
- The importance of σ: Too small will also cause overfitting. Too large will fail to characterize rapid changes in the signal.



Too many Receptive Fields?

- In order to reduce the artificial complexity of the RBF, we need to use fewer number of receptive fields.
- How about using a subset of training data, say $M < N$ of them.
- These M data points will then constitute M receptive field centers.
- How to choose these M points…?
  - At random ➔ **Approach 2.**

$$y_j = \varphi_{ij} = \varphi\left(\left\|\mathbf{x}_i - \mathbf{x}_j\right\|^2\right) = e^{\left(-\frac{M}{d_{\max}^2}\left\|\mathbf{x}_i - \mathbf{x}_j\right\|^2\right)}, \quad i = 1,2,\ldots,N \quad j = 1,2,\ldots,M \qquad \sigma = \frac{d_{\max}}{\sqrt{2M}}$$

Output layer weights are determined as they were in Approach 1, through solving a set of M linear equations!

  - Unsupervised training: K-means ➔ **Approach 3**

    The centers are selected through self organization of clusters, where the data is more densely populated. Determining M is usually heuristic.

Approach 3

K-Means - Unsupervised Clustering - Algorithm

- Choose number of clusters, M
  - Initialize M cluster centers to the first M training data points: $\mathbf{t_k}=\mathbf{x_k}$, $k=1,2,\ldots,M$.
- Repeat
  - At iteration n, group all patterns to the cluster whose center is closest

$$C(\mathbf{x}) = \arg\min_k \|\mathbf{x}(n) - \mathbf{t}_k(n)\|, \quad k = 1,2,\ldots,M$$

$\mathbf{t}_k(n)$: center of kth RBF at n^{th} iteration

  - Compute the centers of all clusters after the regrouping

$$\mathbf{t}_k = \frac{1}{M_k}\sum_{j=1}^{M_k}\mathbf{x}_j$$

New cluster center for k^{th} RBF.

Number of instances in the kth cluster

Instances that are grouped in the kth cluster

- Until there is no change in cluster centers from one iteration to the next.

Determining the Output Weights: LMS Algorithm

Approach 3

- The LMS algorithm is used to minimize the cost function $E(\mathbf{w}) = \frac{1}{2}e^2(n)$ where *e(n)* is the error at iteration *n*: $e(n) = d(n) - z^T(n)\mathbf{w}(n)$

$$\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}(n)} = e(n)\frac{\partial e(n)}{\partial \mathbf{w}} \Rightarrow \frac{\partial e(n)}{\partial \mathbf{w}} = -z(n) \Rightarrow \frac{\partial E(\mathbf{w})}{\partial \mathbf{w}(n)} = -z(n)e(n)$$

- Using the steepest (gradient) descent method: $\mathbf{w}(n+1) = \mathbf{w}(n) + \eta\, z(n)e(n)$

Instance based LMS algorithm pseudocode ***(for single output)***:

Initialize weights, w_j to some small random value, j=1,2,…,M

Repeat

Choose next training pair (**x**, d);

Compute network output at iteration *n*: $y(n) = \sum_{j=1}^{M} w_j \cdot \varphi\left\|\mathbf{x} - t_j\right\| = \mathbf{w}^T \cdot z$

Compute error: $e(n) = d(n) - y(n)$

Update weights: $\mathbf{w}(n+1) = \mathbf{w}(n) + \eta e(n)z(n)$

Until weights converge to a steady set of values

Approach 4: Supervised RBF Training

- This is the most general form.
- All parameters, receptive field centers (first layer weights), output layer weights and spread constants, are learned through iterative supervised training using LMS / gradient descent algorithm.

1. *Linear weights* (output layer)

$$\frac{\partial \mathscr{E}(n)}{\partial w_i(n)} = \sum_{j=1}^{N} e_j(n) G(\|\mathbf{x}_j - \mathbf{t}_i(n)\|_{C_i})$$

$$w_i(n+1) = w_i(n) - \eta_1 \frac{\partial \mathscr{E}(n)}{\partial w_i(n)}, \qquad i = 1, 2, \ldots, m_1$$

2. *Positions of centers* (hidden layer)

$$\frac{\partial \mathscr{E}(n)}{\partial \mathbf{t}_i(n)} = 2 w_i(n) \sum_{j=1}^{N} e_j(n) G'(\|\mathbf{x}_j - \mathbf{t}_i(n)\|_{C_i}) \, \mathbf{\Sigma}_i^{-1} [\mathbf{x}_j - \mathbf{t}_i(n)]$$

$$\mathbf{t}_i(n+1) = \mathbf{t}_i(n) - \eta_2 \frac{\partial \mathscr{E}(n)}{\partial \mathbf{t}_i(n)}, \qquad i = 1, 2, \ldots, m_1$$

3. *Spreads of centers* (hidden layer)

$$\frac{\partial \mathscr{E}(n)}{\partial \mathbf{\Sigma}_i^{-1}(n)} = - \; w_i(n) \sum_{j=1}^{N} e_j(n) G'(\|\mathbf{x}_j - \mathbf{t}_i(n)\|_{C_i}) \mathbf{Q}_{ji}(n)$$

$$\mathbf{Q}_{ji}(n) = [\mathbf{x}_j - \mathbf{t}_i(n)][\mathbf{x}_j - \mathbf{t}_i(n)]^T$$

$$\mathbf{\Sigma}_i^{-1}(n+1) = \mathbf{\Sigma}_i^{-1}(n) - \eta_3 \frac{\partial \mathscr{E}(n)}{\partial \mathbf{\Sigma}_i^{-1}(n)}$$

$$\varepsilon = \frac{1}{2} \sum_{j=1}^{N} e_j^2$$

$$e_j = d_j - \sum_{i=1}^{M} w_k \varphi\left(\left\|\mathbf{x}_j - \mathbf{t}_i\right\|\right)$$

$$G\left(\left\|\mathbf{x}_j - \mathbf{t}_i\right\|_C\right) = \varphi\left(\left\|\mathbf{x}_j - \mathbf{t}_i\right\|\right)$$

G′ represents the first derivative of the function wrt its argument